TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 4: K láng giềng gần nhất

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Khoảng cách và láng giềng **2**
2.1 Ý tưởng . . . 2
2.2 Khoảng cách Euclid . . . 3
2.3 Tự tính bằng numpy . . . 3

3 Chuẩn hoá, bước không được bỏ qua **5**
3.1 Công thức . . . 6
3.2 Đo hậu quả trên tập kiểm tra . . . 6
3.3 Đọc kết quả . . . 7
3.4 Vì sao phải dùng make_pipeline . . . 7

4 Chọn số láng giềng **8**
4.1 Bộ dữ liệu hai vầng trăng . . . 8
4.2 Đo bằng con số . . . 8
4.3 Đọc kết quả . . . 10

5 So với các mô hình đã học **10**

6 KNN cho bài toán hồi quy **12**
6.1 Đọc kết quả . . . 14

7 Bài tập **14**

8 Phụ lục A, bảng tra nhanh **16**

9 Phụ lục B, lỗi thường gặp **16**

1. Mục tiêu bài thực hành

Ba buổi trước, mọi mô hình đều rút dữ liệu huấn luyện thành vài con số rồi bỏ dữ liệu đi. Buổi này làm ngược lại: mô hình giữ nguyên toàn bộ dữ liệu, gặp mẫu mới thì đi hỏi những mẫu cũ giống nó nhất.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Tự tính khoảng cách Euclid và tự tìm k láng giềng gần nhất bằng numpy.
2. Chỉ ra được vì sao một cột có thang đo lớn làm hỏng KNN, và chữa bằng chuẩn hoá.
3. Dùng `make_pipeline` để gộp chuẩn hoá với mô hình cho đúng cách.
4. Chọn số láng giềng k bằng kiểm định chéo mà không nhìn tập kiểm tra.
5. So sánh KNN với hồi quy logistic và Naive Bayes trên dữ liệu có biên cong.
6. Dùng KNN cho bài toán hồi quy và giải thích vì sao nó không ngoại suy được.

Ba bộ dữ liệu

Bài này dùng `data/sinh_vien_mo_rong.csv`, tức bộ sinh viên quen thuộc có thêm một cột chi tiêu hàng tháng. Dùng `data/hai_vang_trang.csv` gồm 300 điểm có ranh giới cong. Và dùng lại `data/gia_nha.csv` của bài 1. Cả ba đều là số liệu mô phỏng dùng cho mục đích giảng dạy.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai04_knn
```

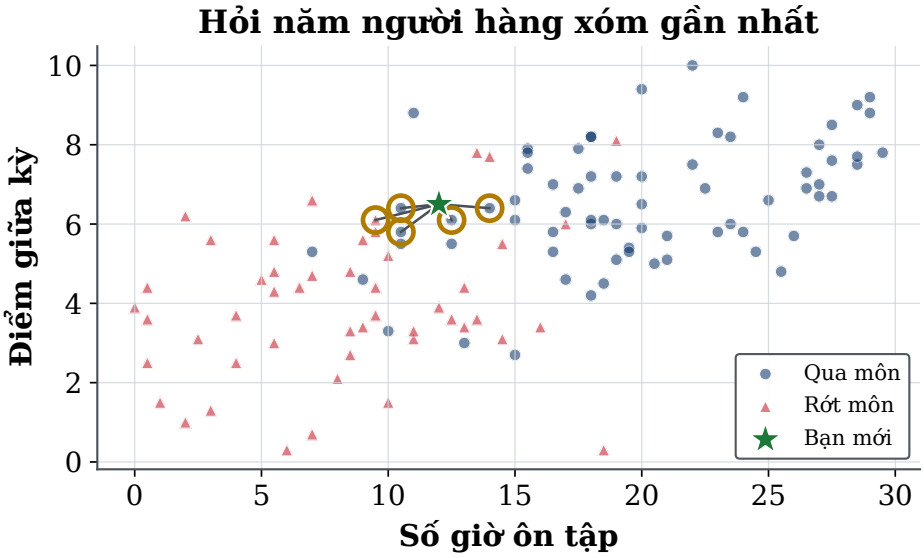
Bảng 1: *Năm tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`e1_khoang_cach.py`	Tự tính khoảng cách và tìm láng giềng
3	`e2_chuan_hoa.py`	Đo hậu quả của việc quên chuẩn hoá
4	`e3_chon_k.py`	Chọn k bằng kiểm định chéo
5	`e4_so_sanh.py`	So KNN với hai mô hình đã học
6	`e5_knn_hoi_quy.py`	KNN cho bài toán hồi quy

2. Khoảng cách và láng giềng

2.1. Ý tưởng

Muốn đoán một bạn mới có qua môn không, ta tìm những bạn cũ giống bạn ấy nhất rồi xem đa số các bạn đó qua hay rớt. Đó là toàn bộ ý tưởng của **K láng giềng gần nhất** *(k-nearest neighbors)*, viết tắt là KNN.



Hình 1: *Một bạn mới ôn 12 giờ, điểm giữa kỳ 6.5, được đánh dấu bằng ngôi sao. Năm bạn cũ gần nhất được khoanh tròn.*

Thuật toán gồm ba bước. Bước một, tính khoảng cách từ mẫu mới tới mọi mẫu cũ. Bước hai, chọn ra k mẫu gần nhất. Bước ba, cho k mẫu đó bỏ phiếu, lớp nào nhiều phiếu hơn thì thắng.

KNN không học gì cả

Bước huấn luyện của KNN chỉ là cất dữ liệu vào bộ nhớ. Mọi phép tính dồn hết sang lúc dự đoán. Vì vậy người ta gọi KNN là mô hình **học lười** *(lazy learning)*. Hệ quả là huấn luyện tức thì nhưng dự đoán chậm, và dữ liệu càng lớn dự đoán càng chậm.

2.2. Khoảng cách Euclid

Khoảng cách giữa hai điểm

$$d(\boldsymbol{x}, \boldsymbol{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2 + \cdots + (x_d - z_d)^2}$$

Đây chính là định lý Pitago mở rộng cho nhiều chiều. Ví dụ bạn P ôn 10 giờ, điểm giữa kỳ 5, còn bạn Q ôn 13 giờ, điểm giữa kỳ 9. Khi đó $d(P, Q) = \sqrt{3^2 + 4^2} = 5$.

2.3. Tự tính bằng numpy

```python
# -*- coding: utf-8 -*-
"""Bước 1: tự tính khoảng cách và tự tìm láng giềng, chưa dùng thư viện.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import numpy as np
import pandas as pd

# Ba bạn: A và C học gần như y hệt nhau, B học khác hẳn
ba_ban = {
```

```
    "A": np.array([5.0, 3.0, 4000.0]),
    "B": np.array([25.0, 9.0, 4100.0]),
    "C": np.array([6.0, 3.5, 7000.0]),
}


def kc(u, v):
    """Khoang cach Euclid giua hai diem."""
    return np.sqrt(((u - v) ** 2).sum())


print("Khoang cach KHI CHUA chuan hoa")
print(f"  d(A, B) = {kc(ba_ban['A'], ba_ban['B']):8.1f}")
print(f"  d(A, C) = {kc(ba_ban['A'], ba_ban['C']):8.1f}")
print()

# Chuẩn hoá bằng trung bình và độ lệch chuẩn của cả bộ dữ liệu
df = pd.read_csv("data/sinh_vien_mo_rong.csv")
X = df[["gio_on", "diem_giua_ky", "chi_tieu"]].to_numpy(dtype=float)
tb = X.mean(axis=0)
dl = X.std(axis=0)
print("Do lech chuan cua ba cot:")
print(f"  gio_on {dl[0]:.2f}   diem_giua_ky {dl[1]:.2f}"
      f"   chi_tieu {dl[2]:.0f}")
print()

chuan = {ten: (v - tb) / dl for ten, v in ba_ban.items()}
print("Khoang cach SAU KHI chuan hoa")
print(f"  d(A, B) = {kc(chuan['A'], chuan['B']):.2f}")
print(f"  d(A, C) = {kc(chuan['A'], chuan['C']):.2f}")
print()

# Tìm 5 láng giềng gần nhất của một bạn mới, chỉ dùng hai cột học tập
X2 = df[["gio_on", "diem_giua_ky"]].to_numpy(dtype=float)
y = df["qua_mon"].to_numpy()
tb2, dl2 = X2.mean(axis=0), X2.std(axis=0)
ban_moi = np.array([12.0, 6.5])
X2c = (X2 - tb2) / dl2
moi_c = (ban_moi - tb2) / dl2
khoang = np.sqrt(((X2c - moi_c) ** 2).sum(axis=1))
gan = np.argsort(khoang)[:5]

print("Ban moi: on 12 gio, diem giua ky 6.5")
print("Nam lang gieng gan nhat:")
for i in gan:
    ket = "qua" if y[i] == 1 else "rot"
    print(f"  gio {X2[i, 0]:5.1f}  diem {X2[i, 1]:4.1f}  "
          f"khoang cach {khoang[i]:.3f}  -> {ket}")
so_qua = int(y[gan].sum())
```

```
print(f"Bo phieu: {so_qua} qua, {5 - so_qua} rot")
print("Du doan:", "qua mon" if so_qua >= 3 else "rot mon")
```

```
python code/e1_khoang_cach.py
```

```
Khoang cach KHI CHUA chuan hoa
  d(A, B) =    102.2
  d(A, C) =   3000.0

Do lech chuan cua ba cot:
  gio_on 7.82   diem_giua_ky 2.09   chi_tieu 1692

Khoang cach SAU KHI chuan hoa
  d(A, B) = 3.85
  d(A, C) = 1.79

Ban moi: on 12 gio, diem giua ky 6.5
Nam lang gieng gan nhat:
  gio  10.5  diem  6.4  khoang cach 0.198  -> qua
  gio  12.5  diem  6.1  khoang cach 0.202  -> qua
  gio  14.0  diem  6.4  khoang cach 0.260  -> qua
  gio   9.5  diem  6.1  khoang cach 0.373  -> rot
  gio  10.5  diem  5.8  khoang cach 0.386  -> qua
Bo phieu: 4 qua, 1 rot
Du doan: qua mon
```

Phần cuối của kết quả tìm được năm láng giềng của bạn mới, gồm 4 bạn qua và 1 bạn rớt, nên mô hình đoán bạn mới qua môn. Chú ý khoảng cách ở đây được tính sau khi chuẩn hoá, và mục 3 sẽ giải thích vì sao bắt buộc phải làm vậy.

Phần đầu của kết quả mới là chỗ đáng suy nghĩ nhất. Bạn A và bạn C học gần như y hệt nhau, chỉ khác nhau ở tiền chi tiêu. Vậy mà khi chưa chuẩn hoá, $d(A, B) = 102.2$ còn $d(A, C) = 3000.0$. Máy kết luận A giống B hơn giống C, chỉ vì A và B tiêu tiền gần bằng nhau.

Cột có thang đo lớn nuốt trọn khoảng cách

Độ lệch chuẩn của cột chi tiêu là 1692 nghìn đồng, trong khi của cột giờ ôn chỉ 7.82 và của cột điểm chỉ 2.09. Chênh 3000 nghìn đồng chi tiêu đè bẹp chênh 20 giờ ôn tập. Hai cột quan trọng nhất gần như vô hình trong phép tính khoảng cách.

Sau khi chuẩn hoá, $d(A, B) = 3.85$ còn $d(A, C) = 1.79$. Lần này C mới là người gần A nhất, đúng như trực giác.

3. Chuẩn hoá, bước không được bỏ qua

3.1. Công thức

Chuẩn hoá z

$$x' = \frac{x - \text{trung bình}}{\text{độ lệch chuẩn}}$$

Sau bước này mọi cột có trung bình 0 và độ lệch chuẩn 1. Nói cách khác, mỗi cột được đo bằng đơn vị riêng của chính nó, và một đơn vị của cột nào cũng có sức nặng như nhau trong phép tính khoảng cách. Trong scikit-learn, việc này do `StandardScaler` đảm nhận.

3.2. Đo hậu quả trên tập kiểm tra

```python
# -*- coding: utf-8 -*-
"""Bước 2: đo hậu quả của việc quên chuẩn hoá trên tập kiểm tra."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_mo_rong.csv")
y = df["qua_mon"]

X3 = df[["gio_on", "diem_giua_ky", "chi_tieu"]]
X_tr, X_te, y_tr, y_te = train_test_split(
    X3, y, test_size=0.25, random_state=17, stratify=y
)


def cham(mo_hinh, cot):
    mo_hinh.fit(X_tr[cot], y_tr)
    return accuracy_score(y_te, mo_hinh.predict(X_te[cot]))


def knn_chuan():
    """KNN co chuan hoa, gop lam mot khoi bang pipeline."""
    return make_pipeline(StandardScaler(), KNeighborsClassifier(7))


ba_cot = ["gio_on", "diem_giua_ky", "chi_tieu"]
hai_cot = ["gio_on", "diem_giua_ky"]

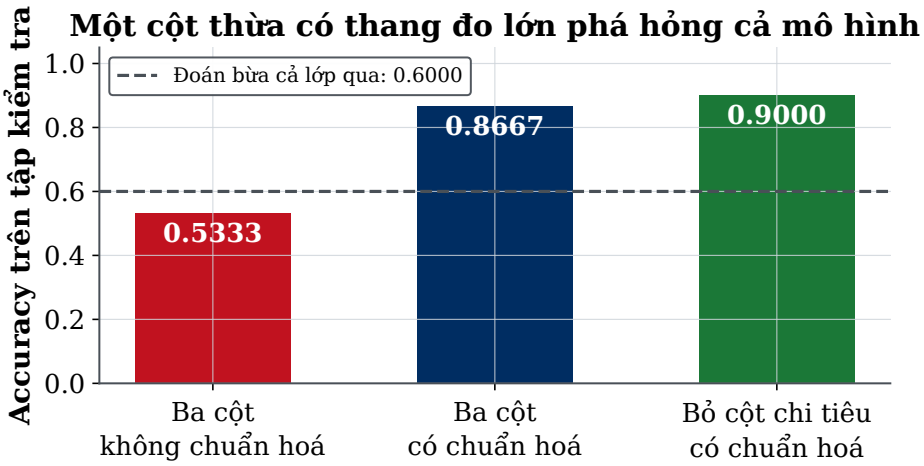
a = cham(KNeighborsClassifier(n_neighbors=7), ba_cot)
b = cham(knn_chuan(), ba_cot)
c = cham(knn_chuan(), hai_cot)

print(f"Doan bua ca lop qua          : {y_te.mean():.4f}")
```

```
print(f"Ba cot, khong chuan hoa      : {a:.4f}")
print(f"Ba cot, co chuan hoa         : {b:.4f}")
print(f"Bo cot chi_tieu, co chuan hoa: {c:.4f}")
```

```
python code/e2_chuan_hoa.py
```

```
Doan bua ca lop qua          : 0.6000
Ba cot, khong chuan hoa      : 0.5333
Ba cot, co chuan hoa         : 0.8667
Bo cot chi_tieu, co chuan hoa: 0.9000
```



Hình 2: *Accuracy trên tập kiểm tra theo ba cách đưa dữ liệu vào KNN. Đường nét đứt là mức đoán bừa cả lớp qua.*

3.3. Đọc kết quả

Ba con số kể một câu chuyện rất rõ.

- Không chuẩn hoá, accuracy chỉ đạt 0.5333. Con số này còn **thấp hơn cả đoán bừa**, vì cứ đoán mọi bạn đều qua đã được 0.6000.
- Chuẩn hoá xong, accuracy vọt lên 0.8667.
- Bỏ hẳn cột chi tiêu rồi chuẩn hoá, accuracy đạt 0.9000.

Hai bài học

Thứ nhất, với KNN chuẩn hoá là **bắt buộc**, không phải tuỳ chọn. Mô hình dựa hoàn toàn vào khoảng cách, nên thang đo quyết định tất cả.
Thứ hai, một cột không liên quan vẫn làm hại mô hình ngay cả khi đã chuẩn hoá. Sau chuẩn hoá, cột chi tiêu có sức nặng ngang hai cột kia, nên nó vẫn góp nhiễu vào khoảng cách. Chọn đúng cột đưa vào mô hình cũng quan trọng như chuẩn hoá.

3.4. Vì sao phải dùng make_pipeline

Trong mã, chuẩn hoá và KNN được gộp làm một bằng `make_pipeline`. Khi gọi `.fit` trên tập học, pipeline tính trung bình và độ lệch chuẩn **chỉ trên tập học**. Khi gọi `.predict` trên tập kiểm tra, nó dùng lại đúng hai con số đó.

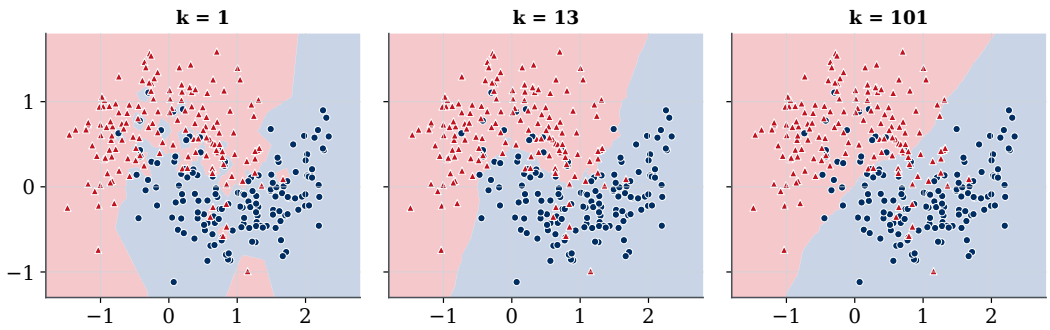
Lỗi rò rỉ dữ liệu hay gặp

Nhiều sinh viên chuẩn hoá toàn bộ dữ liệu trước rồi mới chia tập học và tập kiểm tra. Làm vậy thì trung bình và độ lệch chuẩn đã được tính có cả tập kiểm tra, tức mô hình nhìn trộm đề thi trước khi thi. Dùng `make_pipeline` là cách chắc ăn nhất để không mắc lỗi này.

4. Chọn số láng giềng

4.1. Bộ dữ liệu hai vầng trăng

Tệp `data/hai_vang_trang.csv` có 300 điểm chia đều hai lớp, mỗi lớp nằm trên một vầng trăng khuyết lồng vào nhau. Hai cột `x1` và `x2` có độ lệch chuẩn chênh nhau chỉ khoảng 1.6 lần, nên ở bộ này không chuẩn hoá cũng được.



Hình 3: *Biên quyết định của KNN với ba giá trị k khác nhau trên cùng bộ dữ liệu hai vầng trăng.*

Hình 3 cho thấy k thay đổi hình dạng biên mạnh tới mức nào. Với $k = 1$, biên gập ghềnh và bám theo từng điểm nhiễu, mô hình **học vẹt**. Với $k = 13$, biên cong mềm và đi đúng hình dạng hai vầng trăng. Với $k = 101$, biên gần như thẳng và bỏ qua chi tiết, mô hình **cẩu thả**.

4.2. Đo bằng con số

```python
# -*- coding: utf-8 -*-
"""Bước 3: chọn số láng giềng k bằng kiểm định chéo."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier

df = pd.read_csv("data/hai_vang_trang.csv")
X = df[["x1", "x2"]]
y = df["nhan"]

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

# Hai cột x1 và x2 có thang đo gần nhau, độ lệch chuẩn chỉ chênh khoảng
# 1.6 lần, nên ở bộ này không chuẩn hoá cũng được.
print("Accuracy tren tap hoc va tap kiem tra theo k")
```

```python
print("   k    tap hoc   tap kiem tra")
for k in [1, 5, 13, 31, 101]:
    mh = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
    a_hoc = accuracy_score(y_tr, mh.predict(X_tr))
    a_kt = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {k:3d}   {a_hoc:.4f}     {a_kt:.4f}")
print()

# Chọn k chỉ dựa trên tập học, không nhìn tập kiểm tra
print("Kiem dinh cheo 5 phan tren tap hoc:")
diem = {}
for k in range(1, 52, 2):
    diem[k] = cross_val_score(KNeighborsClassifier(n_neighbors=k),
                              X_tr, y_tr, cv=5).mean()
k_tot = max(diem, key=diem.get)
for k in [1, 5, 9, 13, 17, 25, 51]:
    dau = "  <- tot nhat" if k == k_tot else ""
    print(f"  k = {k:2d}  diem trung binh {diem[k]:.4f}{dau}")
print()

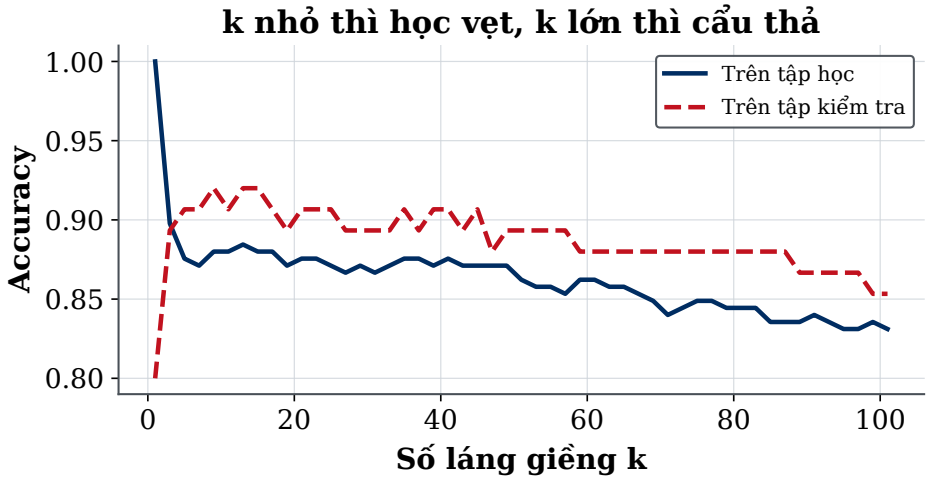
# Chọn xong mới mở tập kiểm tra, và chỉ mở một lần
mh = KNeighborsClassifier(n_neighbors=k_tot).fit(X_tr, y_tr)
print(f"k tot nhat = {k_tot}")
acc = accuracy_score(y_te, mh.predict(X_te))
print(f"Accuracy tren tap kiem tra = {acc:.4f}")
```

```
python code/e3_chon_k.py
```

```
Accuracy tren tap hoc va tap kiem tra theo k
   k    tap hoc   tap kiem tra
   1   1.0000     0.8000
   5   0.8756     0.9067
  13   0.8844     0.9200
  31   0.8667     0.8933
 101   0.8311     0.8533

Kiem dinh cheo 5 phan tren tap hoc:
  k =  1  diem trung binh 0.7778
  k =  5  diem trung binh 0.8444
  k =  9  diem trung binh 0.8444
  k = 13  diem trung binh 0.8667  <- tot nhat
  k = 17  diem trung binh 0.8533
  k = 25  diem trung binh 0.8622
  k = 51  diem trung binh 0.8533

k tot nhat = 13
Accuracy tren tap kiem tra = 0.9200
```



Hình 4: *Accuracy trên tập học và tập kiểm tra khi k chạy từ 1 tới 101.*

4.3. Đọc kết quả

Với $k = 1$, accuracy trên tập học đạt 1.0000 nhưng trên tập kiểm tra chỉ còn 0.8000.

Vì sao một láng giềng đạt tuyệt đối trên tập học

Mỗi điểm của tập học là láng giềng gần nhất của chính nó, nên nó luôn tự bỏ phiếu đúng cho mình. Con số 1.0000 đó không nói lên điều gì, và đây là lý do không bao giờ chấm điểm trên tập học.

Phần kiểm định chéo chọn ra $k = 13$ với điểm trung bình 0.8667, chỉ dựa trên tập học. Chọn xong mới mở tập kiểm tra ra chấm một lần, được 0.9200.

Vì sao không chọn k bằng tập kiểm tra

Nhìn bảng đầu tiên thì $k = 13$ cũng đang cho điểm kiểm tra cao nhất. Nhưng nếu ta chọn k bằng cách nhìn bảng đó, tập kiểm tra đã bị dùng để ra quyết định, và con số 0.9200 không còn là điểm của một bài thi chưa từng thấy.
Kiểm định chéo chia tập học thành 5 phần, lần lượt lấy mỗi phần làm đề thử và 4 phần còn lại để học, rồi lấy trung bình 5 lần chấm. Nhờ vậy ta có một thước đo đáng tin để chọn k mà tập kiểm tra vẫn còn nguyên vẹn.

Chọn k lẻ

Với bài toán hai lớp, k chẵn có thể dẫn tới hoà phiếu, ví dụ hai phiếu qua và hai phiếu rớt. Chọn k lẻ thì chuyện đó không bao giờ xảy ra, nên vòng lặp trong mã chỉ thử các giá trị lẻ.

5. So với các mô hình đã học

```python
# -*- coding: utf-8 -*-
"""Bước 4: đặt KNN cạnh hai mô hình đã học trên dữ liệu có biên cong."""

import pandas as pd
from sklearn.linear_model import LogisticRegression
```

```python
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier

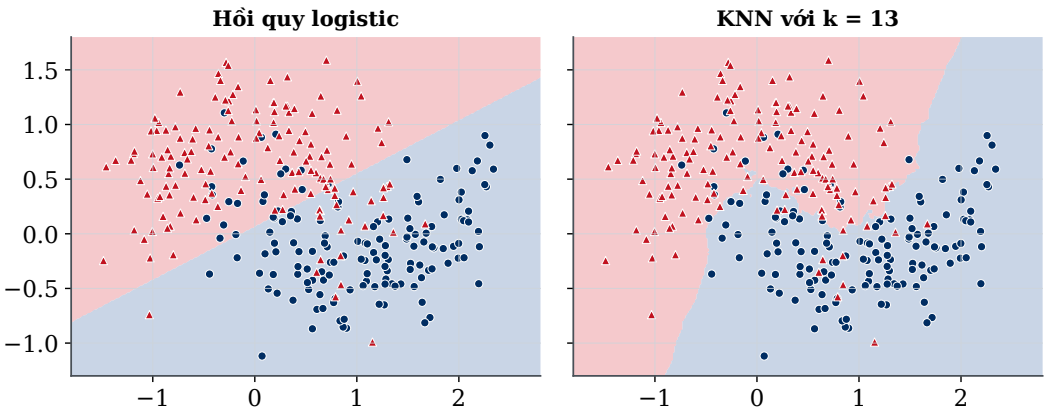
df = pd.read_csv("data/hai_vang_trang.csv")
X = df[["x1", "x2"]]
y = df["nhan"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

cac_mo_hinh = [
    ("Hoi quy logistic", LogisticRegression()),
    ("Naive Bayes     ", GaussianNB()),
    ("KNN, k = 13     ", KNeighborsClassifier(n_neighbors=13)),
]

print("Accuracy tren tap kiem tra, bo hai vang trang")
for ten, mh in cac_mo_hinh:
    mh.fit(X_tr, y_tr)
    print(f"  {ten}: {accuracy_score(y_te, mh.predict(X_te)):.4f}")
```

```
python code/e4_so_sanh.py
```

```
Accuracy tren tap kiem tra, bo hai vang trang
  Hoi quy logistic: 0.8533
  Naive Bayes     : 0.8533
  KNN, k = 13     : 0.9200
```



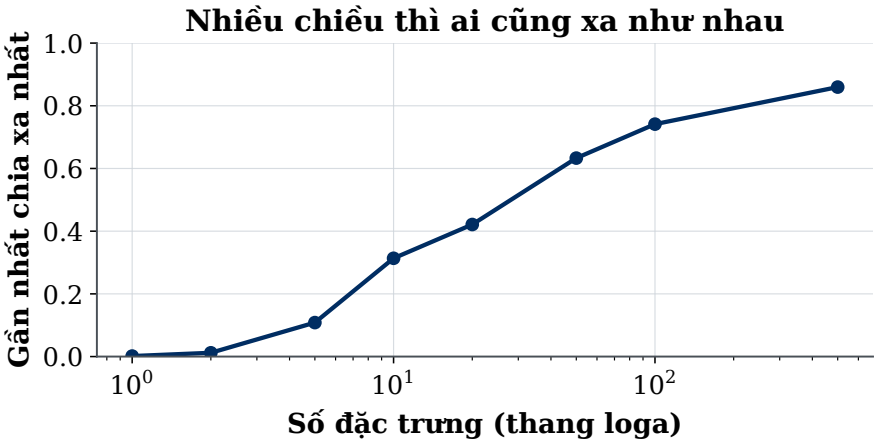
Hình 5: *Biên quyết định của hồi quy logistic và của KNN với $k = 13$ trên bộ hai vầng trăng.*

Hồi quy logistic và Naive Bayes cùng đạt 0.8533, còn KNN đạt 0.9200.

Lý do nằm ở hình 5. Hồi quy logistic chỉ vẽ được một đường thẳng, nên nó cắt ngang qua hai vầng trăng và chấp nhận sai ở hai đầu. KNN không có dạng biên cố định nào, nó tự uốn theo dữ liệu.

Mô hình không tham số

KNN không giả định biên có hình gì, nên gọi là mô hình **không tham số** *(non-parametric)*. Dữ liệu có hình dạng lạ tới đâu nó cũng bám theo được, miễn là có đủ dữ liệu. Cái giá phải trả là KNN cần nhiều dữ liệu hơn và dự đoán chậm hơn.



Hình 6: *Tỷ số giữa khoảng cách gần nhất và xa nhất khi rải ngẫu nhiên 500 điểm trong không gian có số chiều tăng dần.*

Điểm yếu lớn nhất của KNN

Hình 6 rải ngẫu nhiên 500 điểm rồi đo tỷ số giữa khoảng cách gần nhất và xa nhất. Với 2 đặc trưng tỷ số chỉ 0.01, nhưng với 500 đặc trưng nó lên tới 0.86. Khi tỷ số gần 1, láng giềng gần nhất cũng xa gần bằng láng giềng xa nhất, và việc hỏi hàng xóm chẳng còn đáng tin. Đây là lời nguyền số chiều đã gặp ở bài 3. Vì vậy KNN hợp với bài toán ít đặc trưng.

6. KNN cho bài toán hồi quy

KNN không chỉ phân loại được. Muốn đoán một con số, ta tìm k mẫu gần nhất rồi **lấy trung bình** con số của chúng thay vì bỏ phiếu. Trong scikit-learn, lớp tương ứng là `KNeighborsRegressor`.

```python
# -*- coding: utf-8 -*-
"""Bước 5: KNN cho bài toán hồi quy, trên dữ liệu giá nhà của bài 1."""

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor

df = pd.read_csv("data/gia_nha.csv")
X = df[["dien_tich"]]
y = df["gia"]

# Cùng cách chia với bài 1 để so sánh công bằng
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
```

```
                                            random_state=42)

knn = KNeighborsRegressor(n_neighbors=5).fit(X_tr, y_tr)
duong = LinearRegression().fit(X_tr, y_tr)

print("MAE tren tap kiem tra (ty dong)")
mae_knn = mean_absolute_error(y_te, knn.predict(X_te))
mae_duong = mean_absolute_error(y_te, duong.predict(X_te))
print(f"  KNN, k = 5  : {mae_knn:.4f}")
print(f"  Duong thang : {mae_duong:.4f}")
print()

can_moi = pd.DataFrame({"dien_tich": [60.0, 100.0, 150.0, 250.0]})
print("Du doan cho bon can moi (ty dong)")
print("  dien tich     KNN   duong thang")
for dt, a, b in zip(can_moi["dien_tich"], knn.predict(can_moi),
                    duong.predict(can_moi)):
    print(f"  {dt:6.0f} m2   {a:6.2f}   {b:6.2f}")
print()

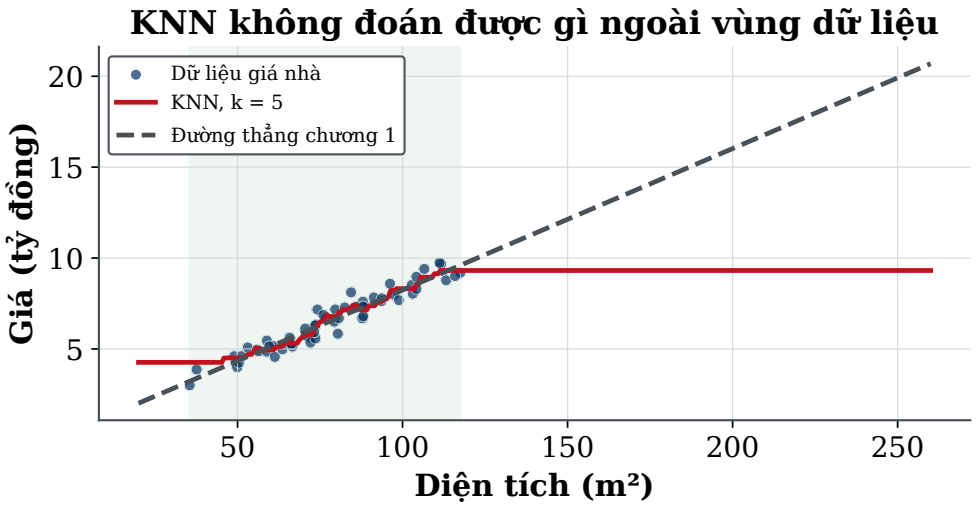
# Năm láng giềng của căn 250 m2 chính là những căn rộng nhất đã thấy
_, chi_so = knn.kneighbors(pd.DataFrame({"dien_tich": [250.0]}))
lg = X_tr.iloc[chi_so[0]]["dien_tich"].tolist()
print("Nam lang gieng cua can 250 m2:", ", ".join(f"{v:.1f}" for v in lg))
```

```
python code/e5_knn_hoi_quy.py
```

```
MAE tren tap kiem tra (ty dong)
  KNN, k = 5  : 0.4030
  Duong thang : 0.2578

Du doan cho bon can moi (ty dong)
  dien tich     KNN   duong thang
      60 m2     4.95     5.12
     100 m2     8.32     8.24
     150 m2     9.32    12.13
     250 m2     9.32    19.91

Nam lang gieng cua can 250 m2: 117.5, 115.9, 111.7, 111.2, 104.2
```



Hình 7: *Đường dự đoán của KNN và của đường thẳng bài 1 trên dữ liệu giá nhà. Vùng nền xanh là khoảng diện tích có dữ liệu.*

6.1. Đọc kết quả

Trên dữ liệu giá nhà, đường thẳng thắng: MAE 0.2578 tỷ so với 0.4030 tỷ của KNN. Điều này hợp lý, vì giá nhà trong bộ dữ liệu tăng gần như thẳng theo diện tích, nên mô hình giả định đường thẳng là giả định đúng và nó tận dụng triệt để.

Bảng dự đoán cho bốn căn mới mới là chỗ đáng chú ý. Với căn 60 m^2 và 100 m^2, hai mô hình đoán gần nhau. Nhưng căn 150 m^2 và căn 250 m^2 đều được KNN đoán đúng một con số 9.32 tỷ.

KNN không ngoại suy được

Năm láng giềng của căn 250 m^2 là năm căn thuộc nhóm rộng nhất trong tập học, từ 104.2 tới 117.5 m^2. Căn 150 m^2 cũng có đúng năm láng giềng đó, nên hai căn nhận cùng một dự đoán. KNN chỉ nói lại những gì đã thấy, không bao giờ đi xa hơn vùng có dữ liệu. Đường thẳng thì cứ kéo dài mãi, và điều đó vừa là sức mạnh vừa là mối nguy như bài 1 đã cảnh báo.

Bài học chung của cả buổi

Không có mô hình nào tốt nhất cho mọi bài toán. Trên hai vầng trăng, KNN thắng vì biên cong. Trên giá nhà, đường thẳng thắng vì quan hệ thật sự gần như thẳng. Chính hình dạng thật của dữ liệu quyết định mô hình nào hợp, và đó là lý do phải luôn vẽ dữ liệu ra nhìn trước.

7. Bài tập

Sinh viên tạo thư mục `baitap04` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap04/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tự viết KNN không dùng thư viện

Dựa vào `e1_khoang_cach.py`, viết hàm `knn_du_doan(X_hoc, y_hoc, x_moi, k)` trả về nhãn dự đoán cho một mẫu mới. Dùng hàm đó dự đoán cho toàn bộ tập kiểm tra của bộ hai vầng trăng với $k = 13$, rồi tính accuracy.
So kết quả với `KNeighborsClassifier` ở mục 4, hai con số phải bằng nhau. Gợi ý, dùng `np.argsort` để lấy chỉ số của k khoảng cách nhỏ nhất.

Bài tập 2: Khoảng cách Manhattan

Chạy lại `e3_chon_k.py` nhưng thêm tham số `metric="manhattan"` vào `KNeighborsClassifier`. Ghi lại k tốt nhất và accuracy trên tập kiểm tra, rồi so với khoảng cách Euclid. Nhận xét một câu: đổi cách đo khoảng cách có làm kết quả khác đi nhiều không?

Bài tập 3: Đổi đơn vị đo

Trên bộ `sinh_vien_mo_rong.csv`, đổi cột `chi_tieu` từ nghìn đồng sang triệu đồng bằng cách chia cho 1000. Chạy KNN với $k = 7$ **không chuẩn hoá** trên ba cột mới.
Accuracy thay đổi thế nào so với 0.5333 ở mục 3? Giải thích vì sao chỉ đổi đơn vị mà kết quả lại khác hẳn. Nếu có chuẩn hoá thì đổi đơn vị có làm kết quả thay đổi không?

Bài tập 4: Láng giềng có trọng số

Thêm tham số `weights="distance"` vào `KNeighborsClassifier`. Khi đó láng giềng càng gần thì phiếu càng nặng.
Chạy lại với $k = 1, 13, 101$ trên bộ hai vầng trăng và so với bảng ở mục 4. Với $k = 101$, trọng số theo khoảng cách có giúp mô hình bớt cẩu thả không? Vì sao?

Bài tập 5: Chọn k cho bài toán hồi quy

Trên dữ liệu giá nhà, dùng kiểm định chéo 5 phần trên tập học để chọn k tốt nhất cho `KNeighborsRegressor`, với k chạy từ 1 tới 20. Gợi ý, truyền `scoring="neg_mean_absolute_error"` vào `cross_val_score`, và nhớ rằng điểm trả về mang dấu âm.
Với k tốt nhất, MAE trên tập kiểm tra có vượt được đường thẳng không?

Bài tập 6: Đo thời gian dự đoán

Tạo bộ dữ liệu ngẫu nhiên có 1000, 10000 và 100000 mẫu với 2 đặc trưng bằng `np.random.default_rng(0).random`. Với mỗi cỡ, đo thời gian `.fit` và thời gian `.predict` cho 1000 mẫu mới bằng mô đun `time`.
Lập bảng kết quả rồi giải thích vì sao KNN được gọi là mô hình học lười. Kết quả thời gian khác nhau giữa các máy là chuyện bình thường, chỉ cần so tương đối giữa các cỡ dữ liệu.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 1 và bài 3 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu KNN thật sự làm gì bên trong hay không.

8. Phụ lục A, bảng tra nhanh

Bảng 2: *Các hàm mới xuất hiện trong bài 4.*

Lệnh	Thư viện	Công dụng
`KNeighborsClassifier`	scikit-learn	KNN cho bài toán phân loại.
`KNeighborsRegressor`	scikit-learn	KNN cho bài toán hồi quy.
`n_neighbors`	scikit-learn	Số láng giềng k, mặc định bằng 5.
`metric`	scikit-learn	Cách đo khoảng cách, mặc định là Euclid.
`weights`	scikit-learn	Đặt `"distance"` để láng giềng gần có phiếu nặng hơn.
`.kneighbors`	scikit-learn	Trả về khoảng cách và chỉ số của các láng giềng.
`StandardScaler`	scikit-learn	Chuẩn hoá mỗi cột về trung bình 0, độ lệch chuẩn 1.
`make_pipeline`	scikit-learn	Gộp chuẩn hoá và mô hình thành một khối.
`cross_val_score`	scikit-learn	Chấm điểm bằng kiểm định chéo, trả về điểm từng lần.
`np.argsort`	numpy	Trả về chỉ số sắp xếp tăng dần, dùng để tìm láng giềng.
`np.sqrt`	numpy	Lấy căn bậc hai.

Bảng 3: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
$d(A, B)$ và $d(A, C)$ khi chưa chuẩn hoá	102.2 và 3000.0
$d(A, B)$ và $d(A, C)$ khi đã chuẩn hoá	3.85 và 1.79
Ba cột, không chuẩn hoá	0.5333
Ba cột, có chuẩn hoá	0.8667
Bỏ cột chi tiêu, có chuẩn hoá	0.9000
Mức đoán bừa trên tập kiểm tra	0.6000
k tốt nhất theo kiểm định chéo	13, điểm 0.8667
Hai vầng trăng, KNN với $k = 13$	0.9200
Hai vầng trăng, logistic và Naive Bayes	cùng bằng 0.8533
Giá nhà, MAE của KNN và của đường thẳng	0.4030 và 0.2578

9. Phụ lục B, lỗi thường gặp

Bảng 4: *Các lỗi hay gặp riêng của bài KNN.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
Accuracy thấp hơn cả đoán bừa	Gần như chắc chắn quên chuẩn hoá, và có một cột thang đo lớn nuốt trọn khoảng cách. Dùng `make_pipeline(StandardScaler(), ...)`.
Accuracy trên tập học bằng 1.0	Đang dùng $k = 1$ và chấm trên tập học. Con số đó vô nghĩa, hãy chấm trên tập kiểm tra.
`ValueError: Expected n_neighbors <= n_samples`	Đặt k lớn hơn số mẫu trong tập học. Giảm k, hoặc kiểm tra lại tập học có bị chia nhầm quá nhỏ không.
Dự đoán rất chậm	KNN phải so mẫu mới với toàn bộ dữ liệu. Với dữ liệu lớn, cân nhắc mô hình khác hoặc giảm số mẫu.
Kết quả đổi khi đổi đơn vị đo	Chưa chuẩn hoá. Sau chuẩn hoá, đổi đơn vị không còn ảnh hưởng tới kết quả.
Điểm kiểm tra đẹp bất thường	Có thể đã chuẩn hoá toàn bộ dữ liệu trước khi chia, hoặc đã chọn k bằng cách nhìn tập kiểm tra.
Đoán mọi mẫu vào cùng một lớp	k quá lớn, gần bằng số mẫu, nên mọi mẫu mới đều có chung một tập láng giềng.
Kết quả khác tài liệu	Quên `random_state=17` hoặc `stratify=y` khi chia dữ liệu.

Một lời khuyên chung

Với KNN, câu hỏi đầu tiên luôn là: các cột đã cùng thang đo chưa? Câu hỏi thứ hai là: có cột nào không liên quan tới thứ cần đoán không? Trả lời xong hai câu này thì phần lớn lỗi của KNN đã được loại bỏ.